CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 5

Streams and World Records

Starts from: SpeedrunCheckpoint4. **Ends at:** SpeedrunCheckpoint5. **Prepares you for:** Lab W14, Part 3: the statistics methods written with streams.

The program's loops now do four kinds of work: counting, collecting, grouping, and finding the best. Each loop takes several lines to say something simple, such as "keep the verified runs." In this checkpoint, each of those jobs becomes a *stream pipeline*, which says what to do in one statement. Along the way, the program fixes the first problem Checkpoint 4 found: leaderboards now count only verified runs, and the report gains a World Records section.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Background: lambdas and method references

A *lambda* is a short method with no name, written right where it is needed. This one takes a run and returns whether it is verified:

```
run -> run.verified()
```

The part before the arrow is the parameter, and the part after it is the value returned. Java accepts a lambda wherever it expects an object of an interface with just one abstract method, such as the test that a stream's `filter` takes.

When a lambda does nothing but call one method, a *method reference* says the same thing more briefly, with two colons. Table 1 lists the ones this checkpoint uses.

Table 1: The method references in this checkpoint, and the lambdas they stand for.

Method reference	The same lambda	Meaning
`Run::verified`	`run -> run.verified()`	Is the run verified?
`Run::player`	`run -> run.player()`	The run's player
`Run::board`	`run -> run.board()`	The run's board
`top::add`	`run -> top.add(run)`	Add the run to `top`
`TreeSet::new`	`() -> new TreeSet<>()`	Make a new, empty `TreeSet`

Step 2: Background: a stream pipeline

A *stream* is a sequence of elements that flows through a series of operations. A pipeline has three parts:

1. **A source,** such as `runs.stream()`, which produces the list's elements one at a time.
2. **Intermediate operations,** each of which produces a new stream: `filter` keeps only the elements that pass a test, and `map` turns each element into something else.
3. **One terminal operation,** which produces the result: `count`, `collect`, `forEach`, or `min`, for example.

A stream never changes the list it came from, and it can be used only once. Nothing happens at all until the terminal operation asks for a result.

Step 3: Count the verified runs

In `main`, right after the line that logs `"Loaded ..."`, add:

```
long verifiedCount = runs.stream().filter(Run::verified).count();
System.out.println(runs.size() + " runs, " + verifiedCount + " verified");
```

```
41 runs, 35 verified
```

Read the pipeline from left to right: take the runs, keep the verified ones, count them. `filter(run -> run.verified())` works exactly the same way; the method reference is just shorter. `count` returns a `long`, so the variable is a `long`.

Step 4: The players, as a pipeline

Replace the loop that fills the `players` set with this, adding an import for `java.util.stream.Collectors`:

```
Set<String> players = runs.stream()
        .map(Run::player)
        .collect(Collectors.toCollection(TreeSet::new));
```

`map` turns each run into its player's name, so the stream now holds 41 names, with repeats. `collect` gathers them into a collection, and `Collectors.toCollection(TreeSet::new)` says which kind: a new `TreeSet`, so the repeats disappear and the names are in order, just as in Checkpoint 4. The simpler `Collectors.toSet()` would also remove repeats, but it promises nothing about order or about which kind of set it builds. Run the program; the players line is the same as before.

Step 5: Grouping, as a pipeline

In Checkpoint 4, `groupByBoard` took a dozen lines to build the map of runs by board. One pipeline does the same. Replace the call to `groupByBoard` with:

```
Map<Board, List<Run>> runsByBoard = runs.stream()
        .collect(Collectors.groupingBy(Run::board, TreeMap::new,
```

```
                Collectors.toList()));
```

Then delete the `groupByBoard` method. `groupingBy` takes three arguments:

- `Run::board`, how to find each run's key;
- `TreeMap::new`, which kind of map to build, so the boards stay in order;
- `Collectors.toList()`, what to do with each board's runs: put them in a list.

Run the program. Apart from the count line from Step 3, the output is the same as at the end of Checkpoint 4, which is the point: the pipeline does what the method did.

Step 6: Runs per game

Changing the third argument of `groupingBy` changes what happens to each group. Add this after the players lines:

```
Map<String, Long> runsPerGame = runs.stream()
        .collect(Collectors.groupingBy(Run::game, TreeMap::new,
                Collectors.counting()));
System.out.println();
System.out.println("Runs per game");
runsPerGame.forEach((game, count) ->
        System.out.printf("  %-38s %2d%n", game, count));
```

```
Runs per game
  Celeste                                 8
  Hollow Knight                           7
  Portal                                  7
  Super Mario 64                         13
  The Legend of Zelda: Ocarina of Time    6
```

This time the key is the game, and `Collectors.counting()` replaces each group with its size, so the values are `Long`s instead of lists. A map has a `forEach` method too, and its lambda takes two parameters, the key and the value, written in parentheses. Super Mario 64 has 13 runs because it has two boards, 120 Star and 16 Star.

Step 7: Leaderboards count only verified runs

In the loop that builds each board's leaderboard, replace the inner `for` loop with a pipeline, and change the title:

```
for (Map.Entry<Board, List<Run>> entry : runsByBoard.entrySet()) {
    Leaderboard<Run> top = new Leaderboard<>(3);
    entry.getValue().stream()
            .filter(Run::verified)
            .forEach(top::add);
    System.out.println();
    printBoard(entry.getKey() + ": top 3 verified", top);
}
```

Run the program. The Portal board now reads:

```
Portal, Inbounds: top 3 verified
 1. Kestrel       10:20.1  2026-08-14
 2. PixelPilot    10:30.1  2026-02-05
 3. Rook          10:52.5  2026-02-08
```

Tangent's unverified 9:12.8 never reaches the leaderboard, because `filter` removes it before `forEach` adds the rest. The same happens on the two other boards that unverified runs used to lead, and no board shows `(unverified)` any more.

Step 8: World records, and Optional

A board's world record is its fastest verified run: the smallest, in the natural order from Checkpoint 3. Add this after the leaderboard loop, with imports for `java.util.Comparator` and `java.util.Optional`:

```
System.out.println();
System.out.println("World Records");
for (Map.Entry<Board, List<Run>> entry : runsByBoard.entrySet()) {
    Optional<Run> record = entry.getValue().stream()
            .filter(Run::verified)
            .min(Comparator.naturalOrder());
    if (record.isPresent()) {
        Run run = record.get();
        System.out.printf("  %-43s %-10s %10s  %s%n", entry.getKey(),
                run.player(), RunTime.format(run.time()), run.date());
    } else {
        System.out.printf("  %-43s no verified runs%n", entry.getKey());
    }
}
```

```
World Records
  Celeste, Any%                              Kestrel       26:41.6  2026-02-14
  Hollow Knight, Any%                        NovaByte      33:34.8  2026-01-31
  Portal, Inbounds                           Kestrel       10:20.1  2026-08-14
  Super Mario 64, 120 Star                   Sprocket    1:38:42.1  2025-12-28
  Super Mario 64, 16 Star                    Rook          15:54.4  2026-01-16
  The Legend of Zelda: Ocarina of Time, 100% Sprocket    3:54:51.2  2026-09-13
```

- **`Comparator.naturalOrder()`** tells `min` to use `Run`'s `compareTo`, so a tie still goes to the earlier run, as Kestrel's Celeste record shows.
- **`min` returns an `Optional<Run>`,** not a `Run`, because the stream might be empty: a board whose runs are all unverified has no record. An `Optional` is a box that holds either one value or nothing. `isPresent()` says which, and `get()` takes the value out.
- **Always check before calling `get`.** On an empty `Optional`, which prints as `Optional.empty`, `get` throws:

  ```
  Exception in thread "main" java.util.NoSuchElementException: No value present
  ```

Every board in our data has a verified run, so the `else` branch never runs here, but a program should not depend on its data being lucky.

Your project should now match SpeedrunCheckpoint5.

Check yourself

1. Write `Run::player` as a lambda.
2. In `runs.stream().filter(Run::verified).count()`, which part is the source, which is an intermediate operation, and which is the terminal operation?
3. In Step 6, what did changing `Collectors.toList()` to `Collectors.counting()` change about the map?
4. Why does `min` return an `Optional<Run>` instead of a `Run`?
5. Tangent's 9:12.8 is the fastest Portal time in the file. Which line keeps it off the Portal leaderboard and out of the world records?

Answers

1. `run -> run.player()`.
2. `runs.stream()` is the source, `filter(Run::verified)` is an intermediate operation, and `count()` is the terminal operation.
3. Each value became the number of runs in the group, a `Long`, instead of the list of runs, so the map's type changed from `Map<String, List<Run>>` to `Map<String, Long>`.
4. The stream it searches might be empty, as it would be for a board with no verified runs. An `Optional` can say "nothing" safely, where returning `null` would invite a `NullPointerException`.
5. `.filter(Run::verified)`, in both pipelines. Tangent's run is not verified, so it is removed before the leaderboard or `min` ever sees it.